

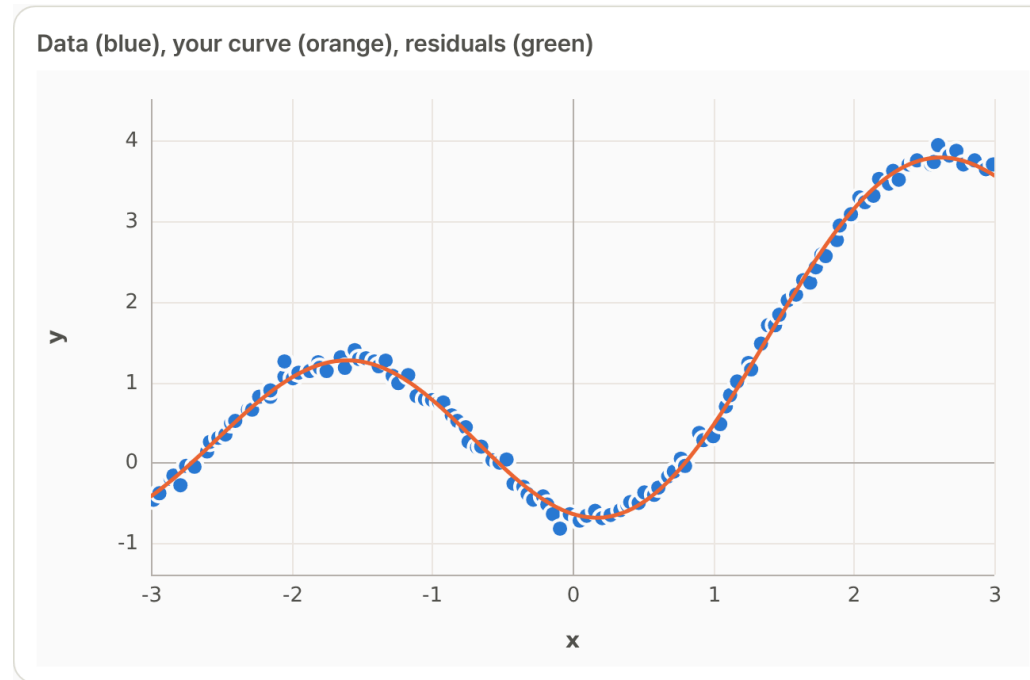
Data & AI

5. Artificial Neural Networks

Dasaem Jeong · Art & Technology, Sogang University

Last Time

Gradient descent on eight bumps



- Measure the slope of every knob, step against it, and repeat
- The model was a formula we wrote: eight bumps, one knob for each height

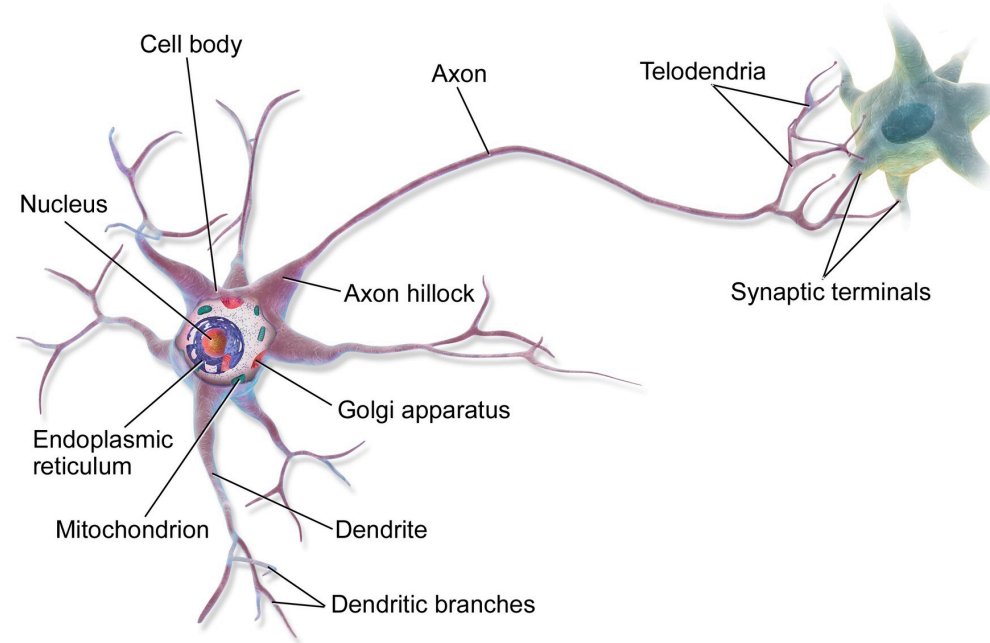
Today

jdasam.github.io/mas1004/demos/lecture05-neuron-patch

- Compute the output of one artificial neuron by hand
- Put neurons side by side into a layer, and write the layer as a matrix multiplication
- Stack layers and train every knob with gradient descent

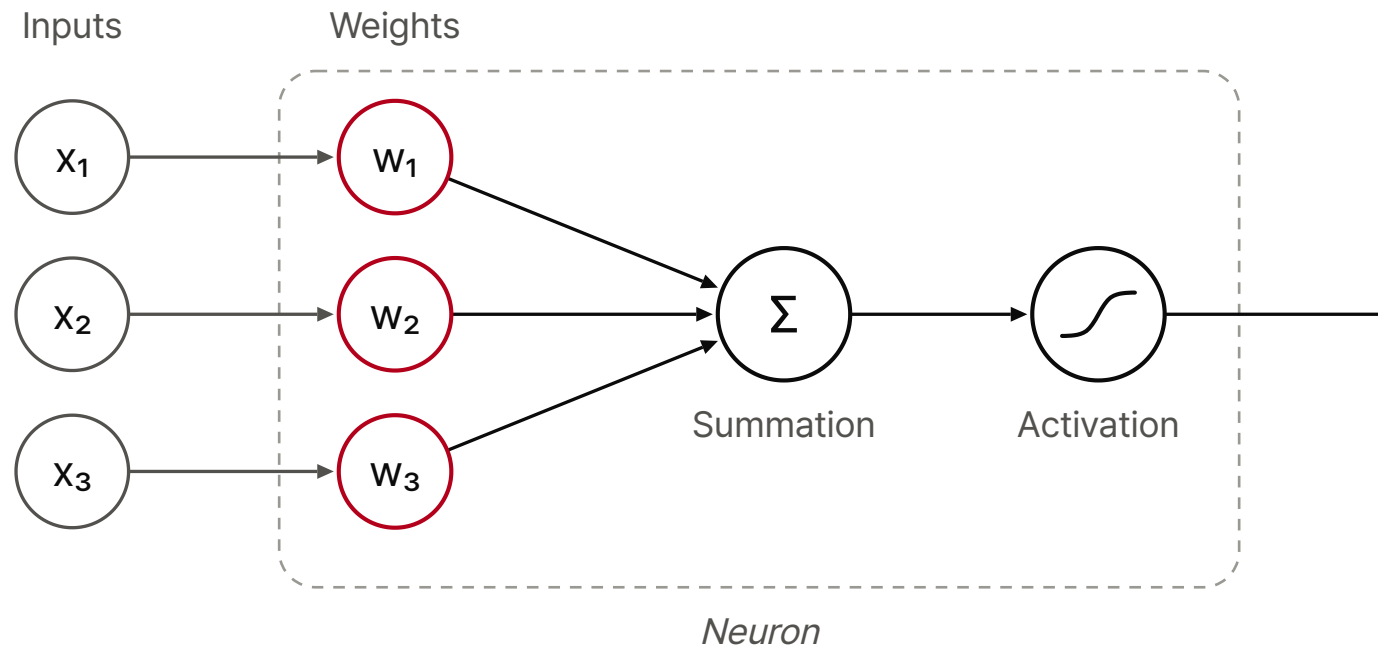
One neuron

Neuron



- A neuron takes in signals from other neurons through its dendrites
- When the input passes a threshold, it sends a signal down its axon

Artificial Neuron



- Takes n numbers as input, one for each feature of a sample
- Gives one number as output

Input Vector

x: a list of numbers, y: the number to predict

	A student	A movie viewer	A neighborhood
x_1	March mock exam score	Rating of The Odyssey	Number of households
x_2	June mock exam score	Rating of Spider-Man	Average house price
x_3	September mock exam score	Rating of Hope	Subway stations within 1 km
y	CSAT score	Rating of KPop Demon Hunters	Number of parking spaces

- The numbers in one vector can measure different kinds of things

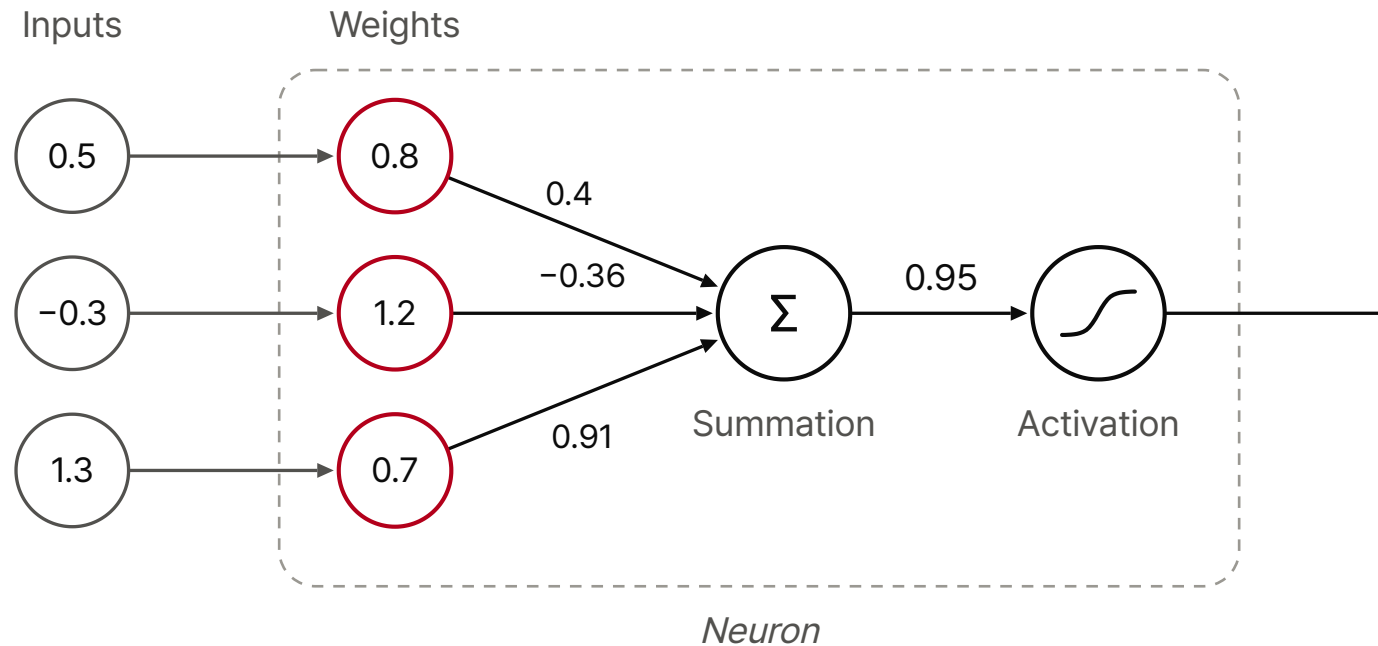
Data Samples

Ratings from three viewers

Viewer	x_1 The Odyssey	x_2 Spider-Man	x_3 Hope	y KPop Demon Hunters
A	4	3	5	4
B	4	5	3	2
C	5	2	5	5

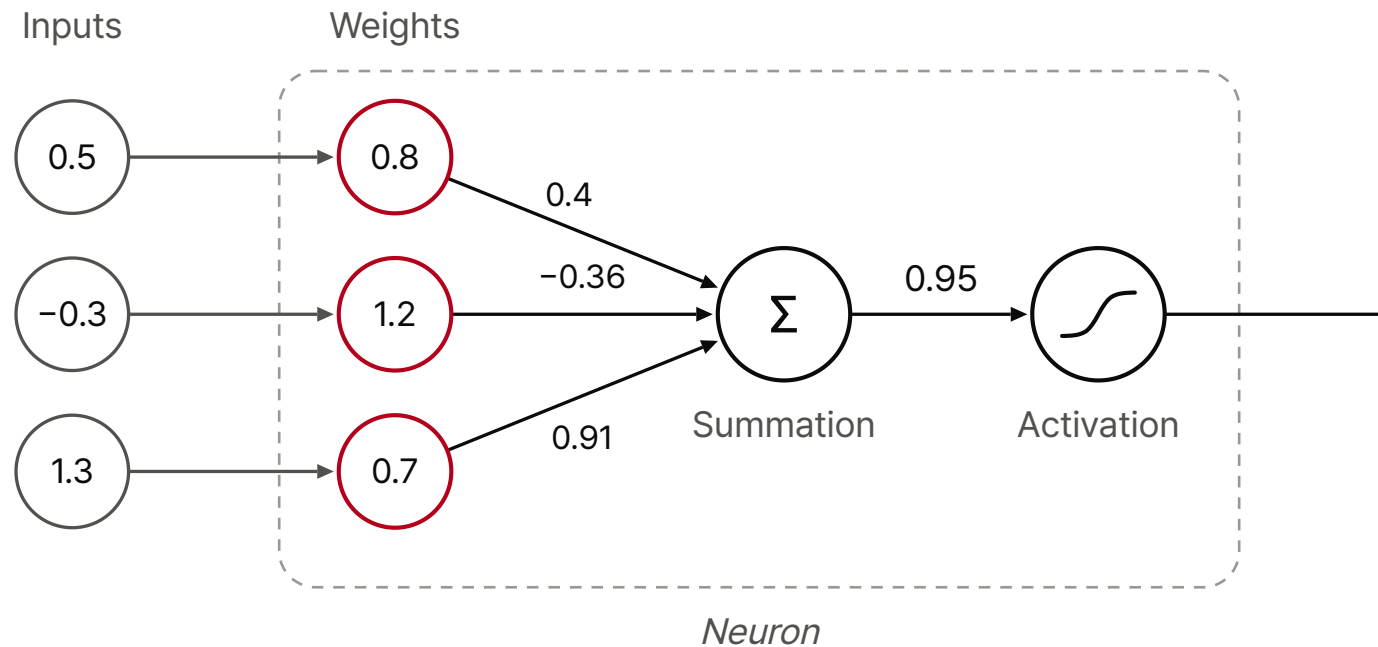
- One sample is one row, with the same inputs in the same order

Weighted Sum



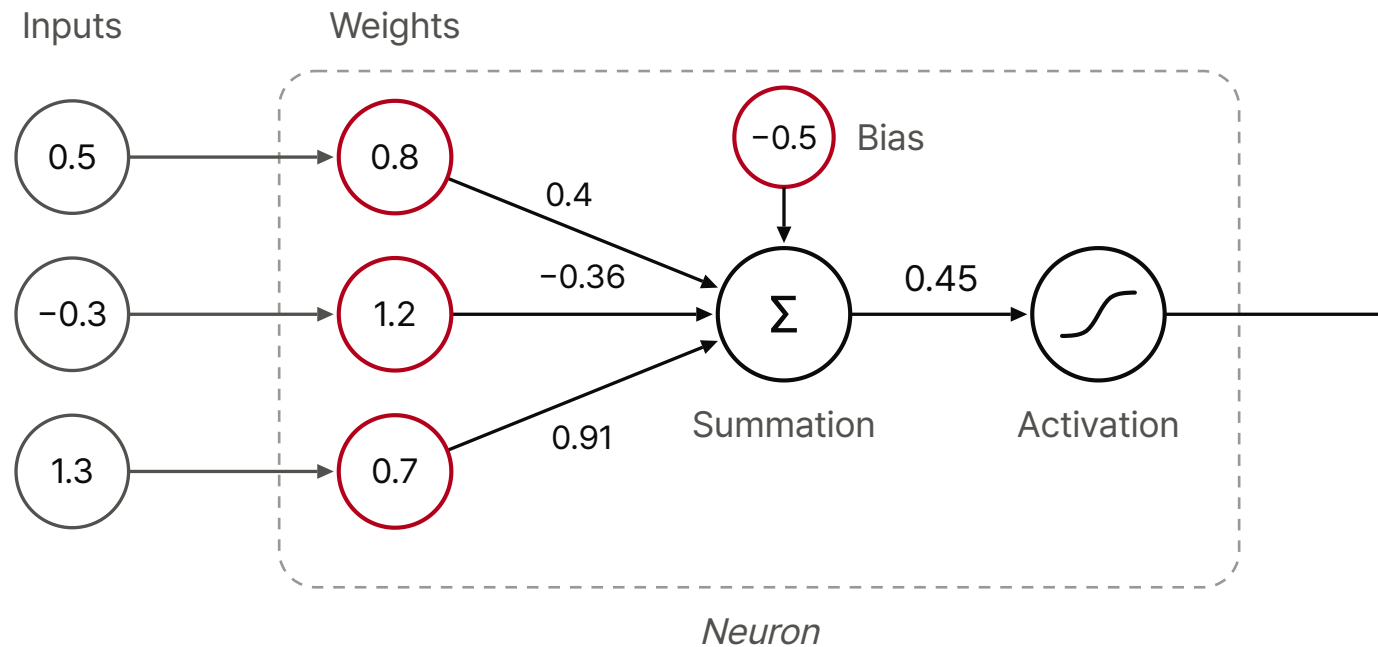
- Multiply each input by its weight, then add: $0.4 - 0.36 + 0.91 = 0.95$
- A weight sets how much an input counts, and with which sign

Weighted Sum as a Matrix Multiplication



$$\begin{bmatrix} 0.5 & -0.3 & 1.3 \end{bmatrix} \times \begin{bmatrix} 0.8 \\ 1.2 \\ 0.7 \end{bmatrix} = 0.5 \times 0.8 + (-0.3) \times 1.2 + 1.3 \times 0.7 = 0.95$$

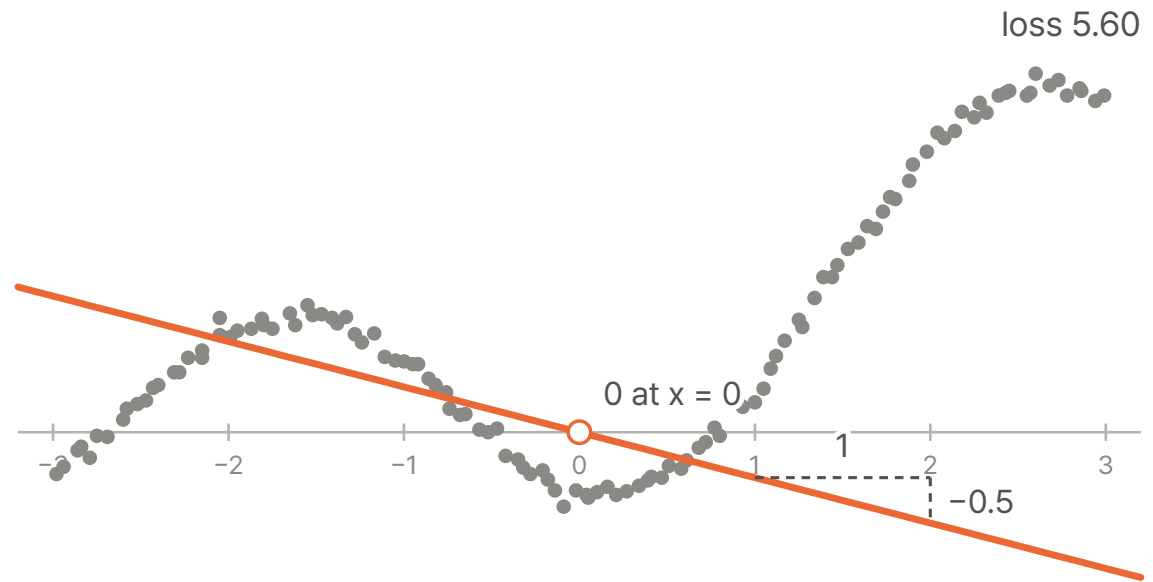
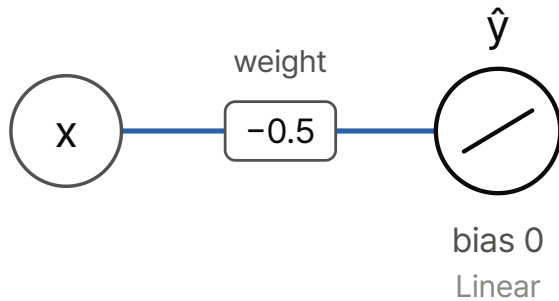
Bias



- A number added to the weighted sum, whatever the inputs are: $0.95 - 0.5 = 0.45$
- Each neuron has its own bias

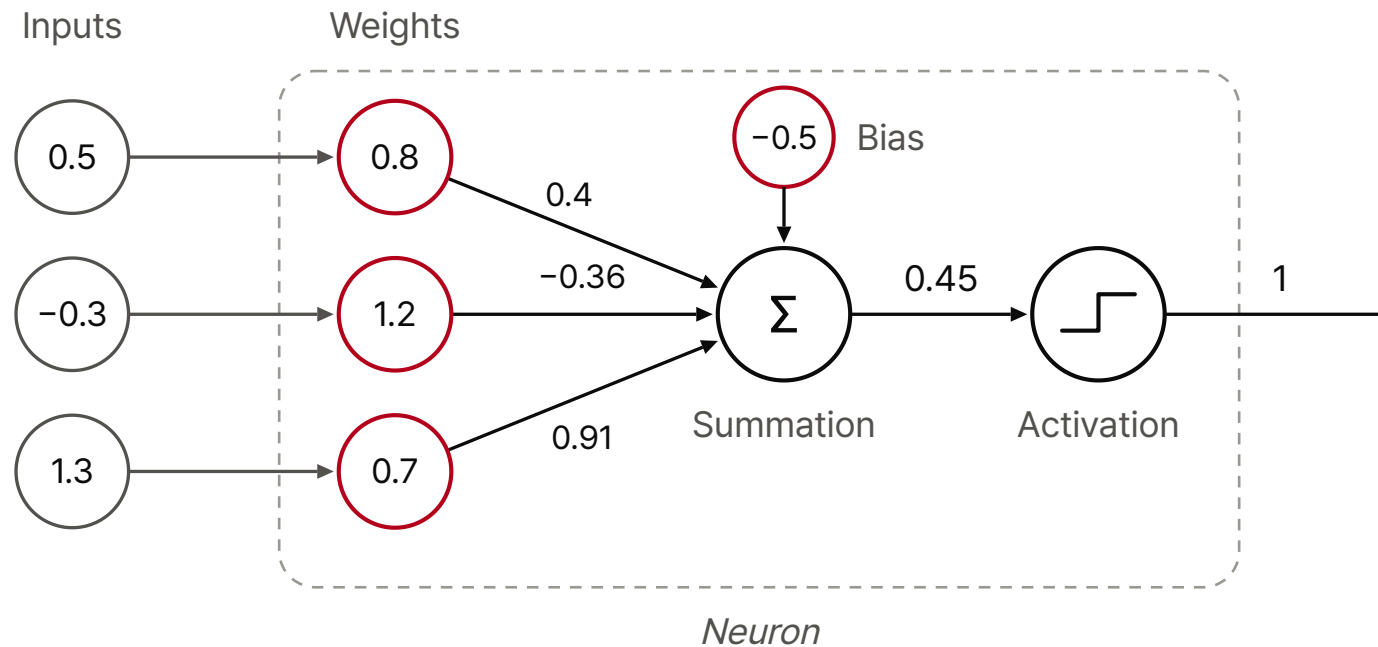
One Neuron Without Activation

Demo, preset "1 neuron"



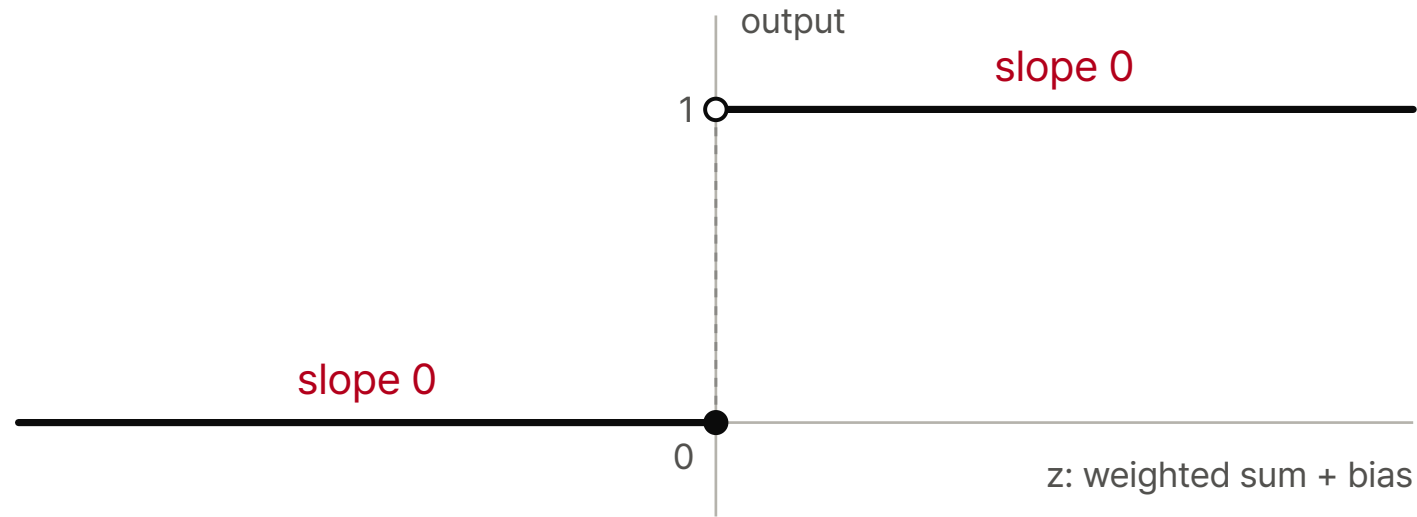
- $\hat{y} = -0.5 \cdot x + 0$: the line $y = a \cdot x + b$ from lecture 3
- The weight is the slope a , and the bias is the value at $x = 0$

Activation Function



- Step function: output 1 when the sum is above 0, otherwise 0
- A threshold, like the one in a real neuron

Slope of the Step Function

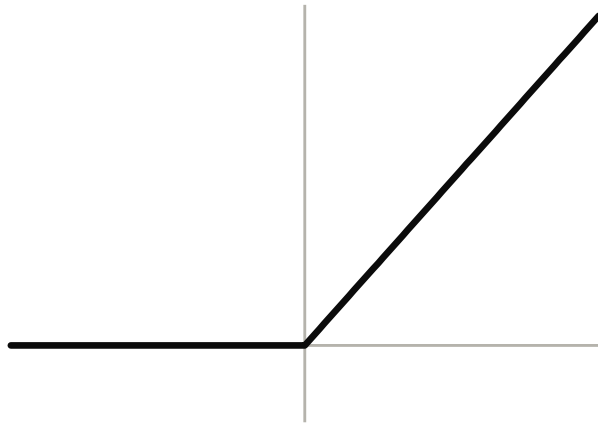


- Nudge a weight a little: the sum moves, but the output stays 0 or 1
- The loss does not change, so gradient descent from lecture 4 gets no direction

Activation Functions

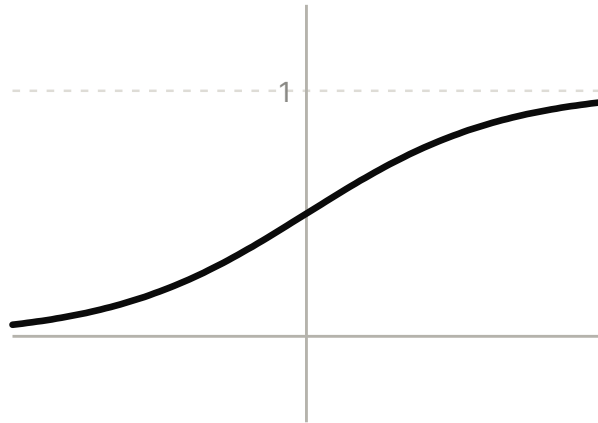
ReLU, sigmoid, tanh

ReLU



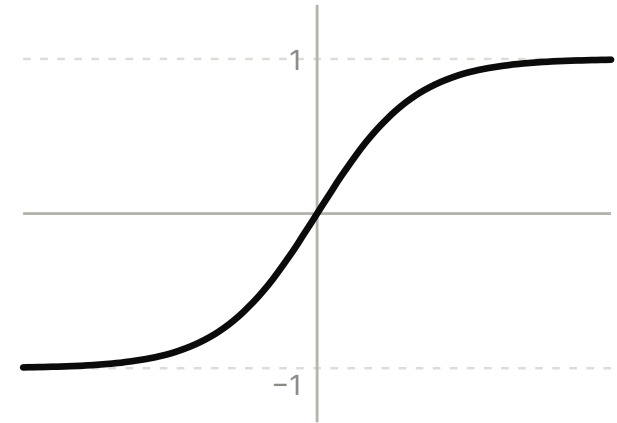
$\max(0, z)$
range: 0 and up

sigmoid



$1 / (1 + \exp(-z))$
range: 0 to 1

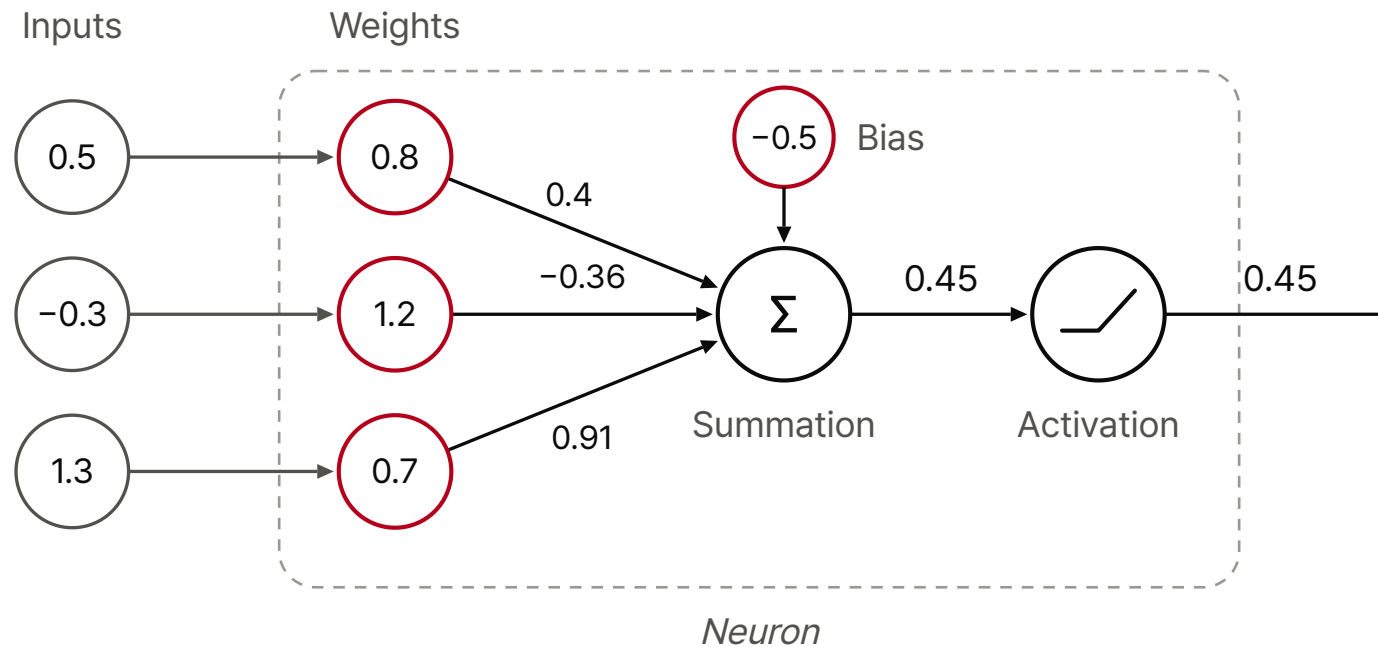
tanh



$\tanh(z)$
range: -1 to 1

- Each has a slope that is not zero over a range of z , so gradient descent can use it

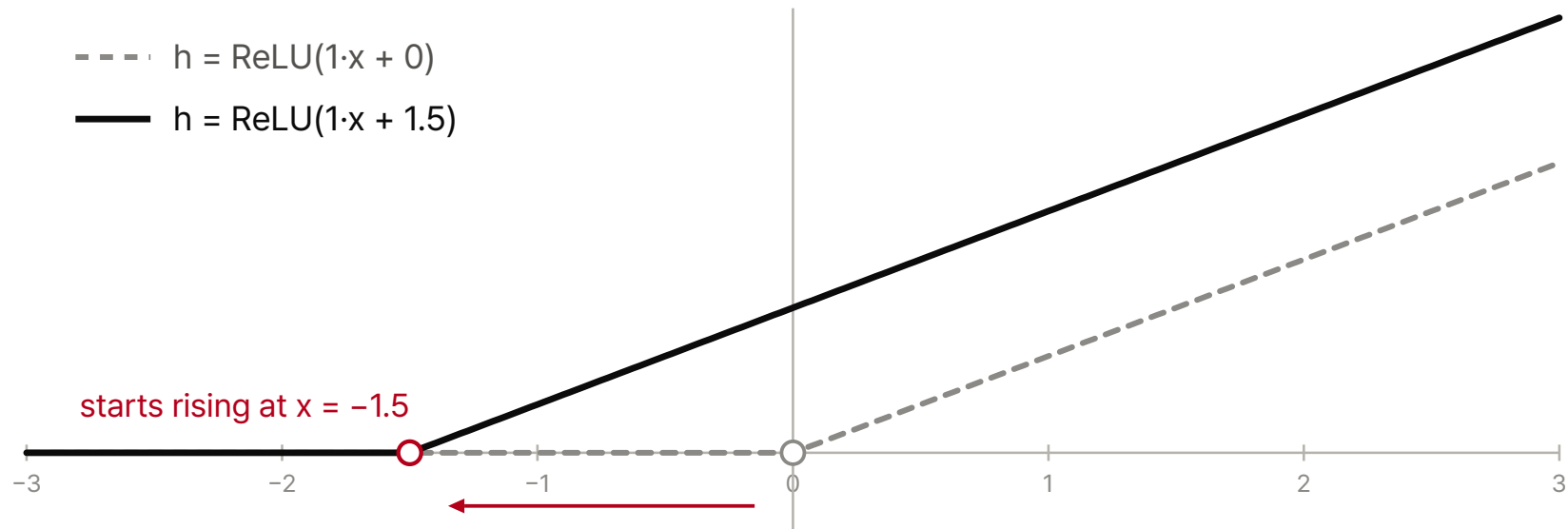
ReLU



- $\text{ReLU}(z) = \max(0, z)$: 0.45 stays 0.45, and a negative sum becomes 0

Role of the Bias

Demo, preset "1 ReLU"



- $h = \text{ReLU}(w \cdot x + b)$ stays 0 until $w \cdot x + b$ reaches 0, then rises in a straight line
- That happens at $x = -b / w$, so the bias moves it left or right

A layer

Matrix Multiplication

Row i of the first matrix and column j of the second give entry (i, j)

$$\begin{bmatrix} \boxed{1} & \boxed{2} \\ 3 & 4 \\ 5 & 6 \end{bmatrix}_{3 \times 2} \times \begin{bmatrix} \boxed{1} & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2 \times 3} = \begin{bmatrix} \boxed{9} & & \\ & & \\ & & \end{bmatrix}_{3 \times 3}$$

$$1 \times 1 + 2 \times 4 = 9$$

- Multiply the matching numbers, then add

Matrix Multiplication

Row i of the first matrix and column j of the second give entry (i, j)

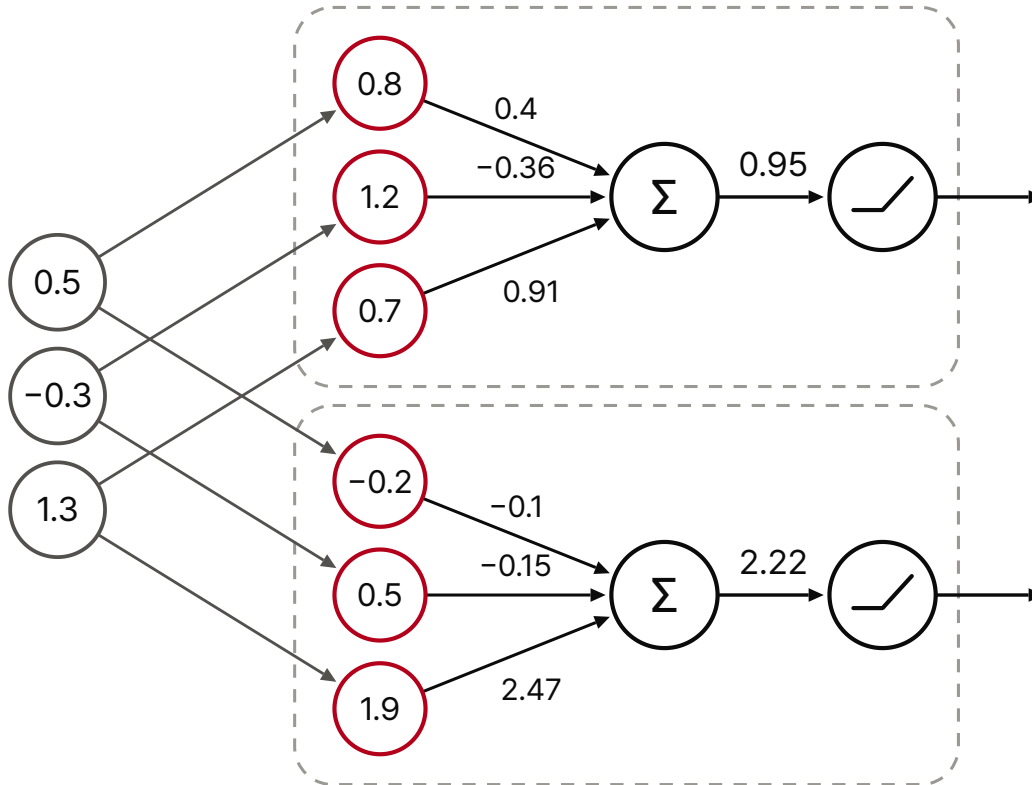
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ \boxed{5} & \boxed{6} \end{bmatrix}_{3 \times 2} \times \begin{bmatrix} 1 & 2 & \boxed{3} \\ 4 & 5 & \boxed{6} \end{bmatrix}_{2 \times 3} = \begin{bmatrix} 9 & 12 & 15 \\ 19 & 26 & 33 \\ 29 & 40 & \boxed{51} \end{bmatrix}_{3 \times 3}$$

$$5 \times 3 + 6 \times 6 = 51$$

- The columns of the first must match the rows of the second: $(3 \times 2) \times (2 \times 3) = 3 \times 3$
- The order matters: $B \times A$ is a different product

Several Neurons

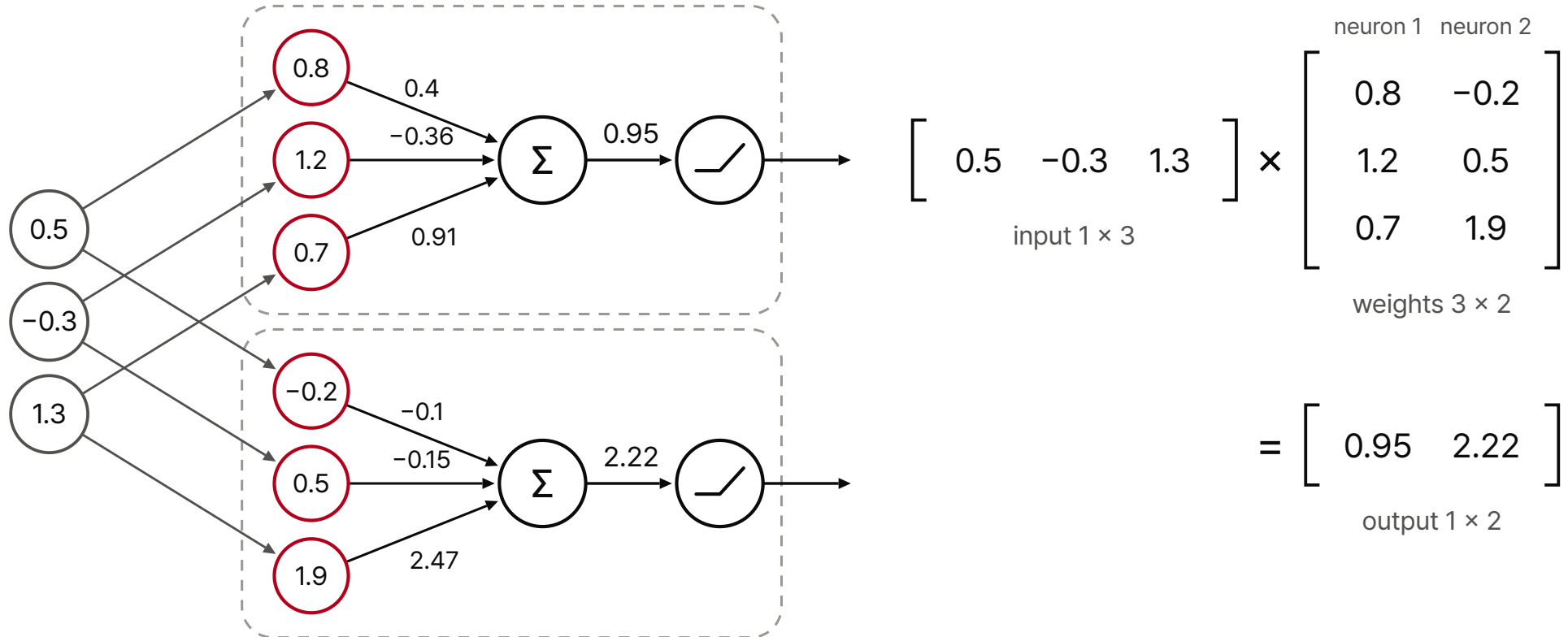
Same inputs, different weights, biases left out



- A layer: neurons that take the same inputs

A Layer as a Matrix Multiplication

Same inputs, different weights, biases left out



- Each column of the weight matrix holds one neuron's weights

Many Samples at Once

The same two neurons, two input samples

$$\begin{array}{c} \text{sample 1} \\ \text{sample 2} \end{array} \begin{array}{ccc} x_1 & x_2 & x_3 \\ \left[\begin{array}{ccc} 0.5 & -0.3 & 1.3 \\ -1.2 & -0.5 & -0.1 \end{array} \right] \end{array} \times \begin{array}{cc} \text{neuron 1} & \text{neuron 2} \\ \left[\begin{array}{cc} 0.8 & -0.2 \\ 1.2 & 0.5 \\ 0.7 & 1.9 \end{array} \right] \end{array} = \begin{array}{c} \text{sample 1} \\ \text{sample 2} \end{array} \begin{array}{cc} \text{neuron 1} & \text{neuron 2} \\ \left[\begin{array}{cc} 0.95 & 2.22 \\ -1.63 & -0.20 \end{array} \right] \end{array}$$

2×3 3×2 2×2
samples $\times$ inputs inputs $\times$ neurons samples $\times$ neurons

- Each row of the input is one sample, and the same row of the result is its output
- One matrix multiplication computes the layer for every sample

Shapes in a Layer

Three points (x, y) into a layer of three neurons

$$\begin{array}{c} \begin{array}{cc} & \begin{array}{cc} x & y \end{array} \\ \begin{array}{l} \text{sample 1} \\ \text{sample 2} \\ \text{sample 3} \end{array} & \begin{bmatrix} 0.12 & -1.4 \\ 2.1 & 3.1 \\ -3.0 & -2.7 \end{bmatrix} \end{array} \quad \times \quad \begin{array}{c} \begin{array}{ccc} & \begin{array}{ccc} \text{neuron 1} & \text{neuron 2} & \text{neuron 3} \end{array} \\ \begin{array}{l} x \\ y \end{array} & \begin{bmatrix} 0.23 & -1.2 & 1.8 \\ -0.34 & 2.5 & 0.5 \end{bmatrix} \end{array} = \begin{array}{c} \begin{array}{ccc} & \begin{array}{ccc} \text{neuron 1} & \text{neuron 2} & \text{neuron 3} \end{array} \\ \begin{array}{l} \text{sample 1} \\ \text{sample 2} \\ \text{sample 3} \end{array} & \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix} \end{array} \end{array}$$

3×2 2×3 3×3
samples $\times$ inputs inputs $\times$ neurons samples $\times$ neurons

- (samples $\times$ inputs) $\times$ (inputs $\times$ neurons) = samples $\times$ neurons

Shapes in a Layer

Three points (x, y) into a layer of three neurons

$$\begin{array}{c} \begin{array}{cc} & x & y \\ \text{sample 1} & \left[\begin{array}{cc} 0.12 & -1.4 \end{array} \right] \\ \text{sample 2} & \left[\begin{array}{cc} 2.1 & 3.1 \end{array} \right] \\ \text{sample 3} & \left[\begin{array}{cc} -3.0 & -2.7 \end{array} \right] \end{array} \times \begin{array}{c} \begin{array}{ccc} & \text{neuron 1} & \text{neuron 2} & \text{neuron 3} \\ x & \left[\begin{array}{ccc} 0.23 & -1.2 & 1.8 \end{array} \right] \\ y & \left[\begin{array}{ccc} -0.34 & 2.5 & 0.5 \end{array} \right] \end{array} \end{array} = \begin{array}{c} \begin{array}{ccc} & \text{neuron 1} & \text{neuron 2} & \text{neuron 3} \\ \text{sample 1} & \left[\begin{array}{ccc} 0.50 & -3.64 & -0.48 \end{array} \right] \\ \text{sample 2} & \left[\begin{array}{ccc} -0.57 & 5.23 & 5.33 \end{array} \right] \\ \text{sample 3} & \left[\begin{array}{ccc} 0.23 & -3.15 & -6.75 \end{array} \right] \end{array} \end{array}$$

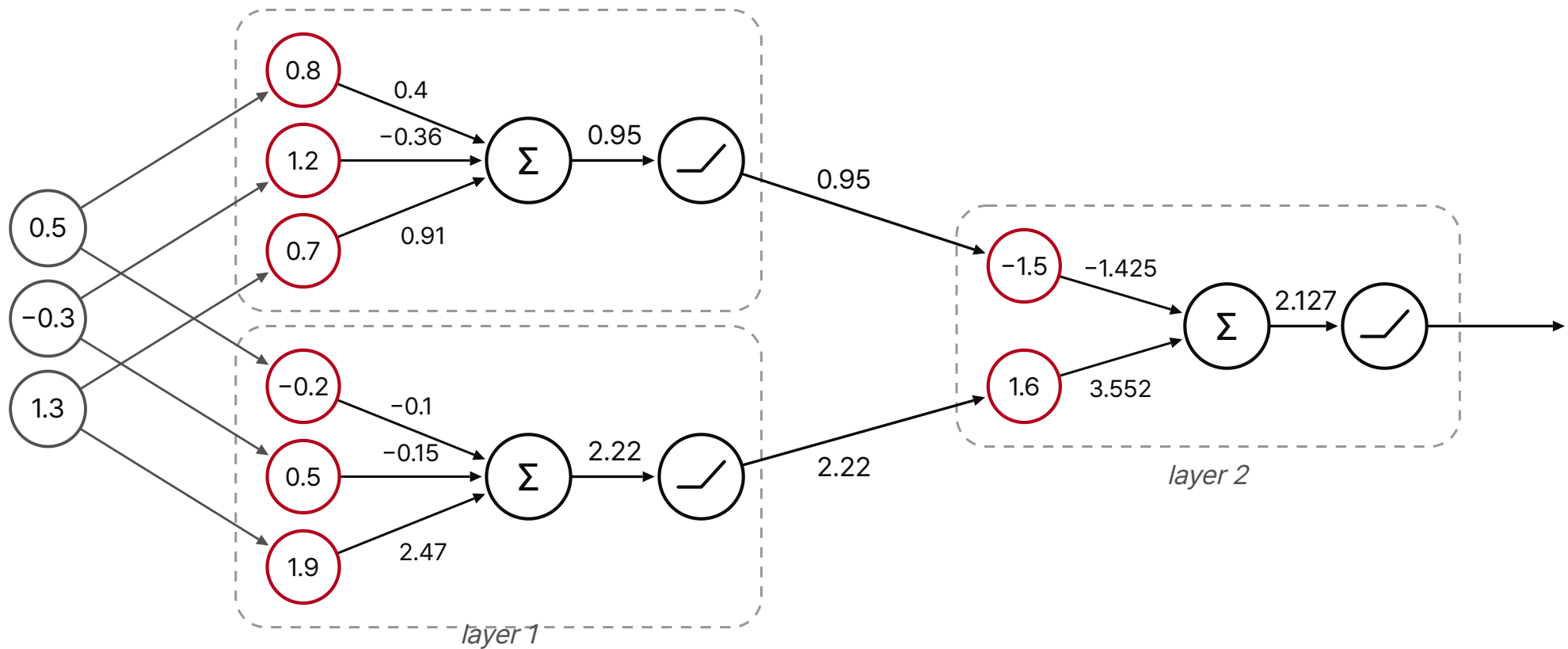
3×2 2×3 3×3
samples × inputs inputs × neurons samples × neurons

- (samples × inputs) × (inputs × neurons) = samples × neurons
- Sample 1, neuron 1: $0.12 \times 0.23 + (-1.4) \times (-0.34) = 0.50$

Stacking layers

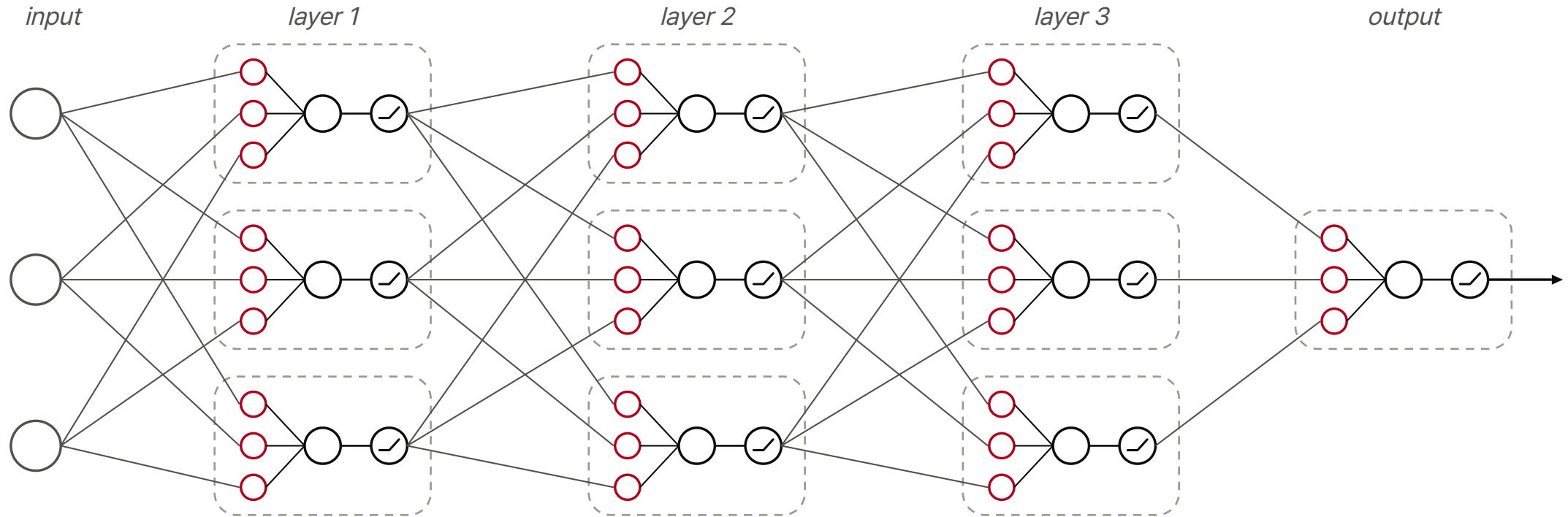
Stacking Neurons

The outputs of layer 1 are the inputs of layer 2



- $0.95 \times (-1.5) + 2.22 \times 1.6 = 2.127$

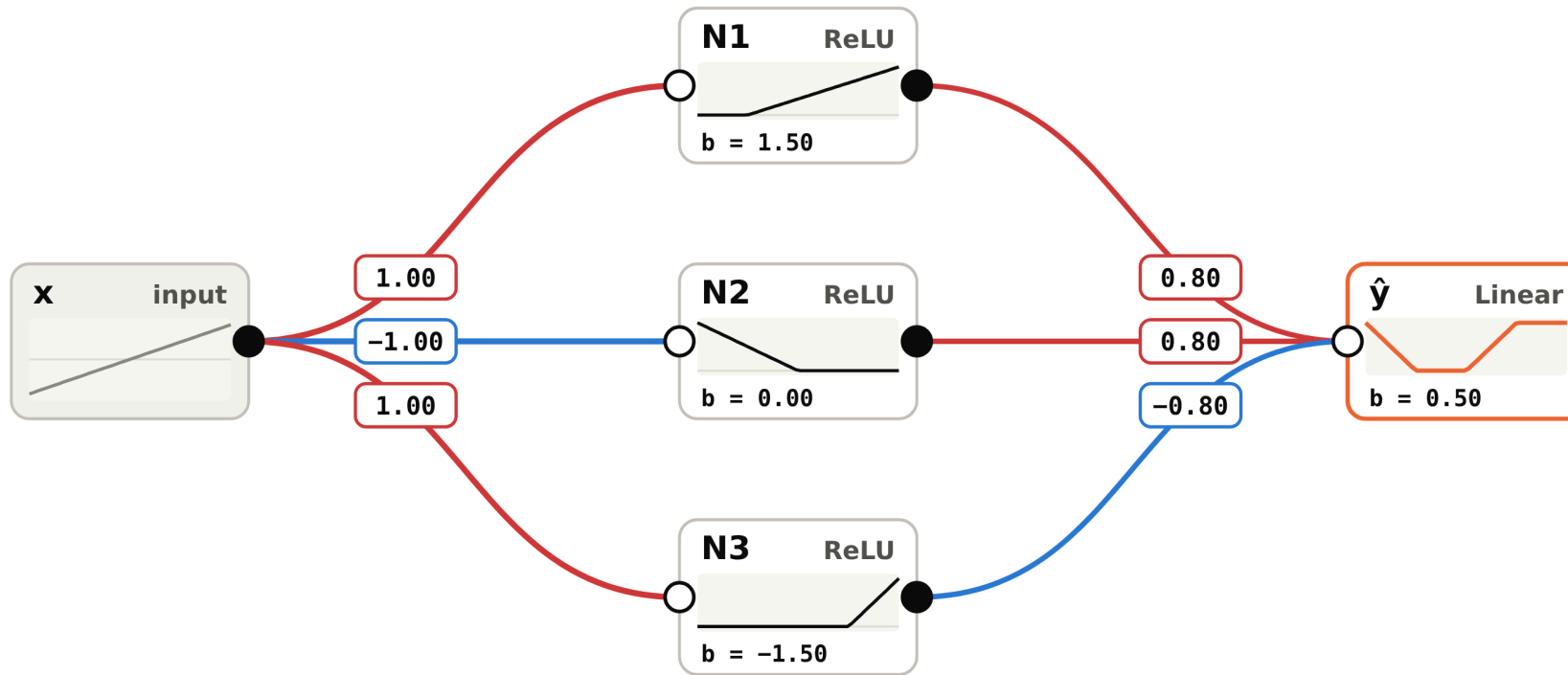
Deep Neural Network



- Each layer: a matrix multiplication, a bias for each neuron, then an activation
- Deep: many of these layers stacked

Knobs in a Patch

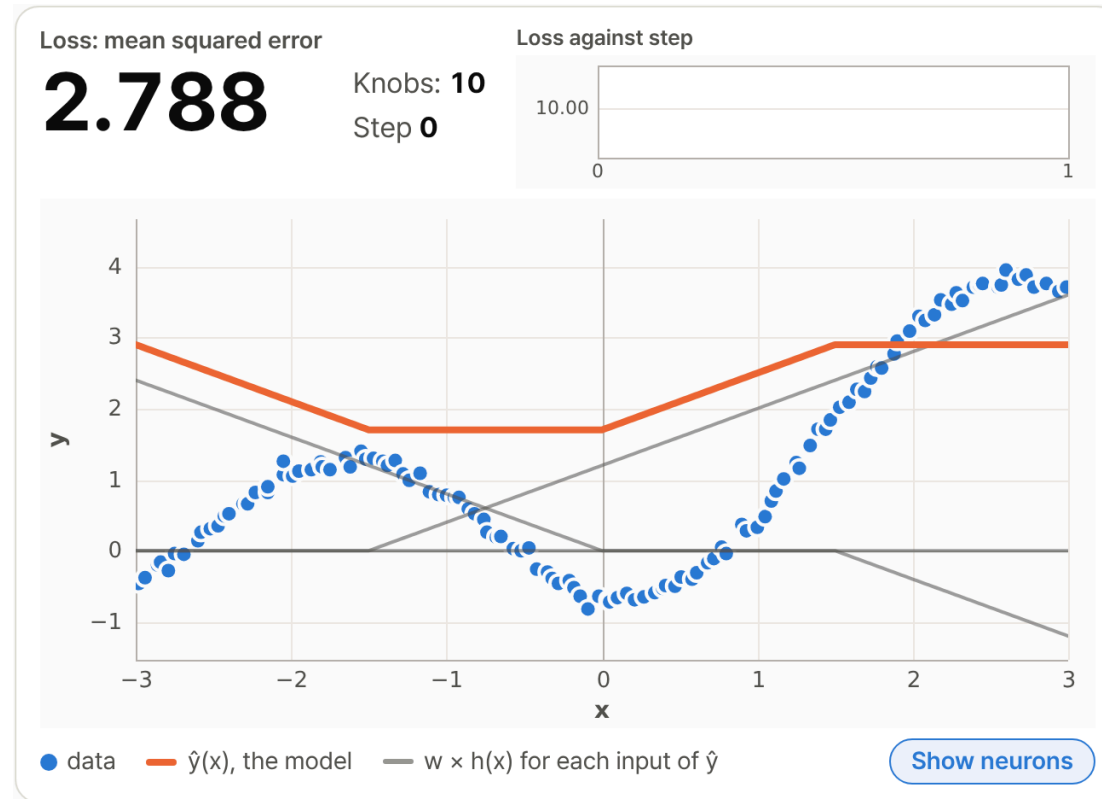
Demo, preset "3 ReLU"



- Every wire is a weight, and every neuron has a bias, $\hat{y}$ included
- 6 weights and 4 biases: 10 knobs

Three ReLU Neurons

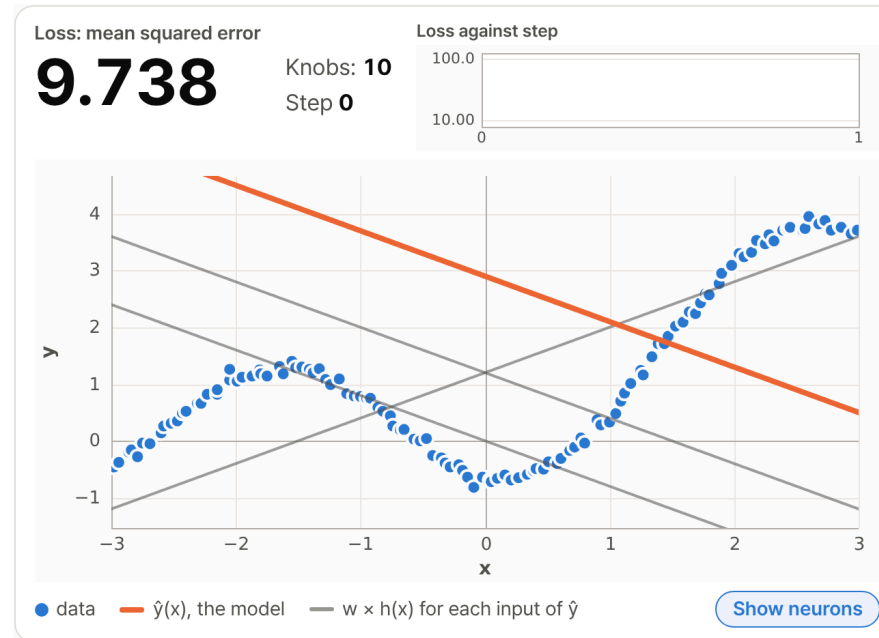
Demo, preset "3 ReLU", knobs turned by hand



- Orange: $\hat{y}$. Gray, with Show neurons: each neuron's output times its weight into $\hat{y}$
- $\hat{y}$ changes its slope where a neuron's sum $w \cdot x + b$ crosses 0: $x = -1.5, 0, 1.5$

Three Neurons Without Activation

Demo, preset "3 Linear"

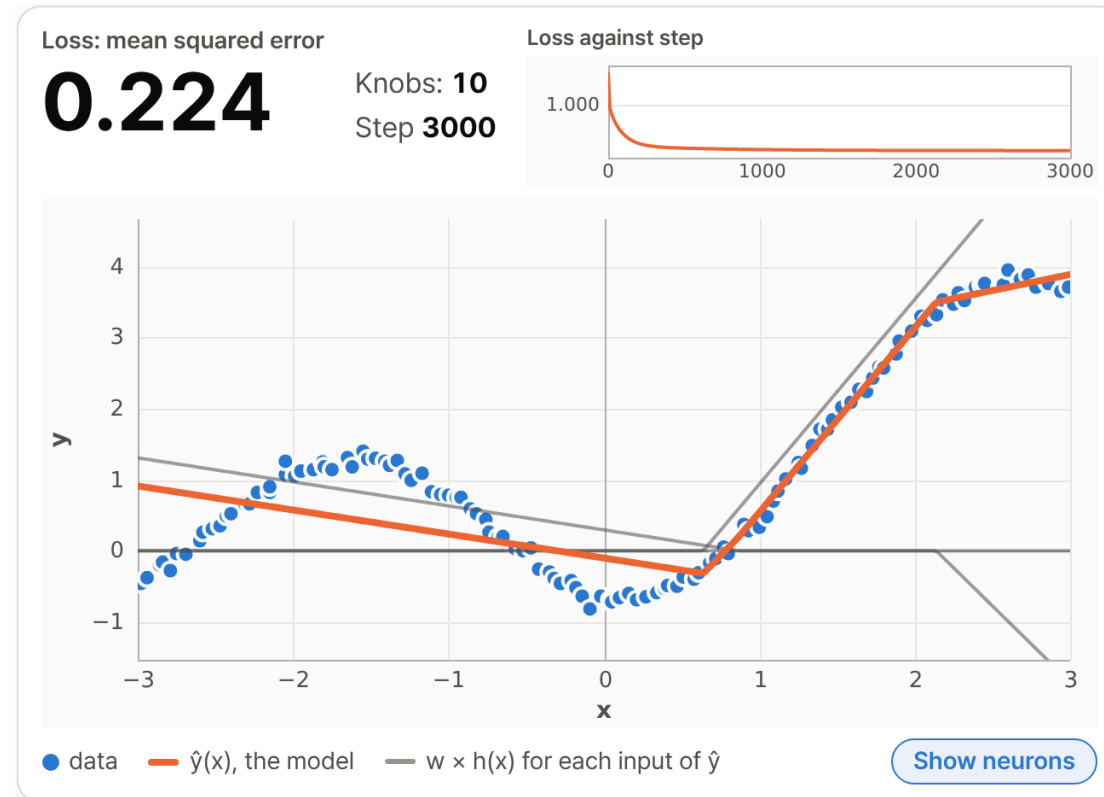


$$\hat{y} = 0.8(x + 1.5) + 0.8(-x) - 0.8(x - 1.5) + 0.5 = -0.8x + 2.9$$

- A weighted sum of lines plus a bias is still a line
- Without activation, a stack of layers works like a single layer

Training Every Knob

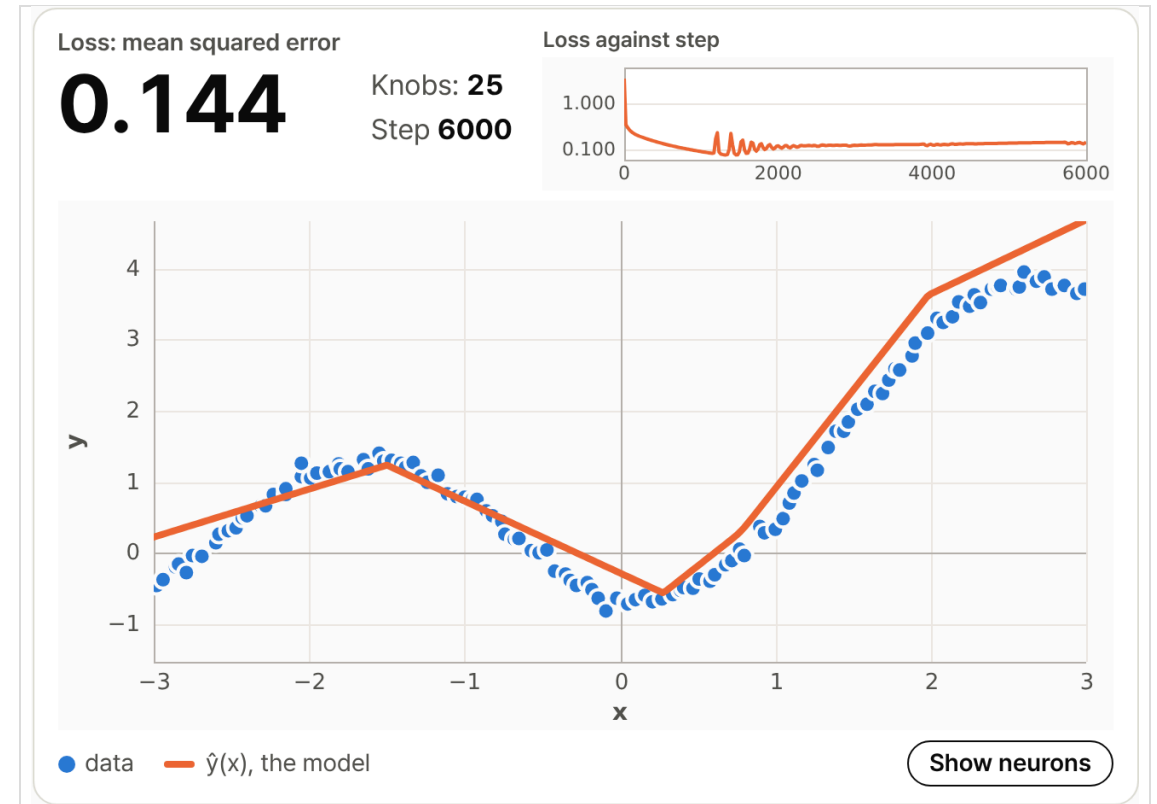
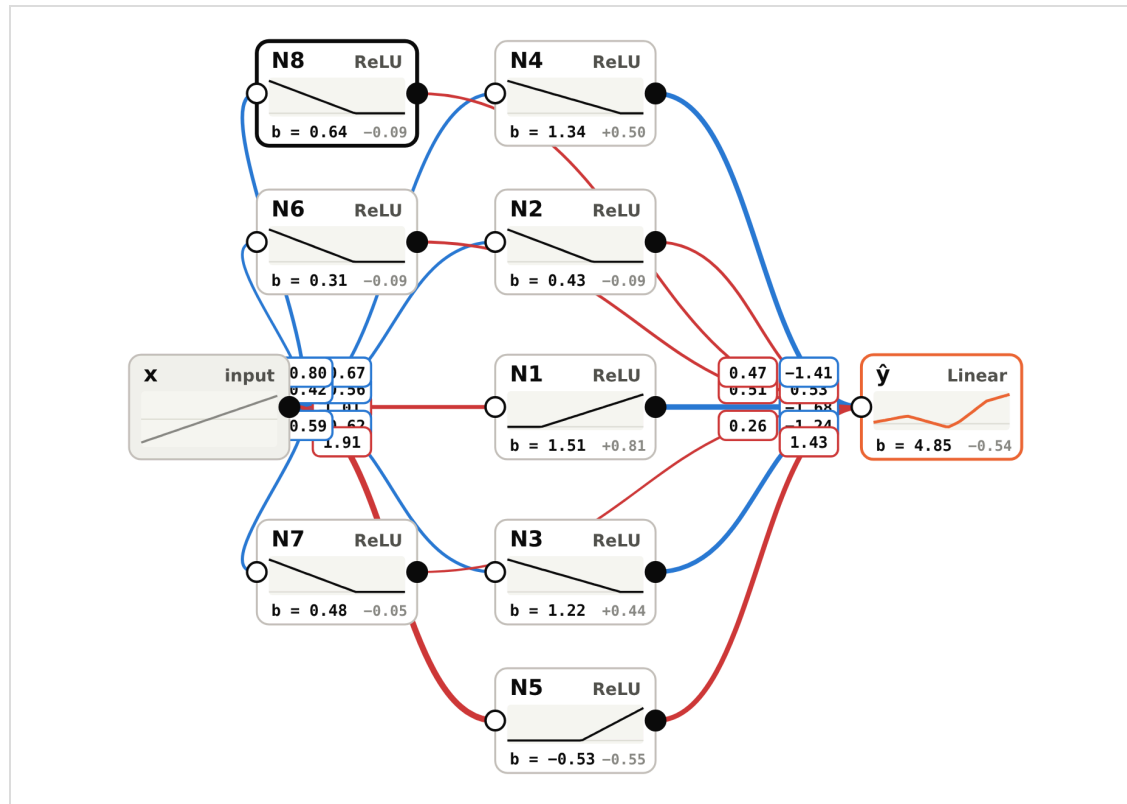
Demo, preset "3 ReLU", Run



- Run repeats lecture 4's rule for every weight and every bias: $w \leftarrow w - lr \times \text{slope}$
- The places where $\hat{y}$ changes its slope move, and each straight piece gets a new slope

More Neurons

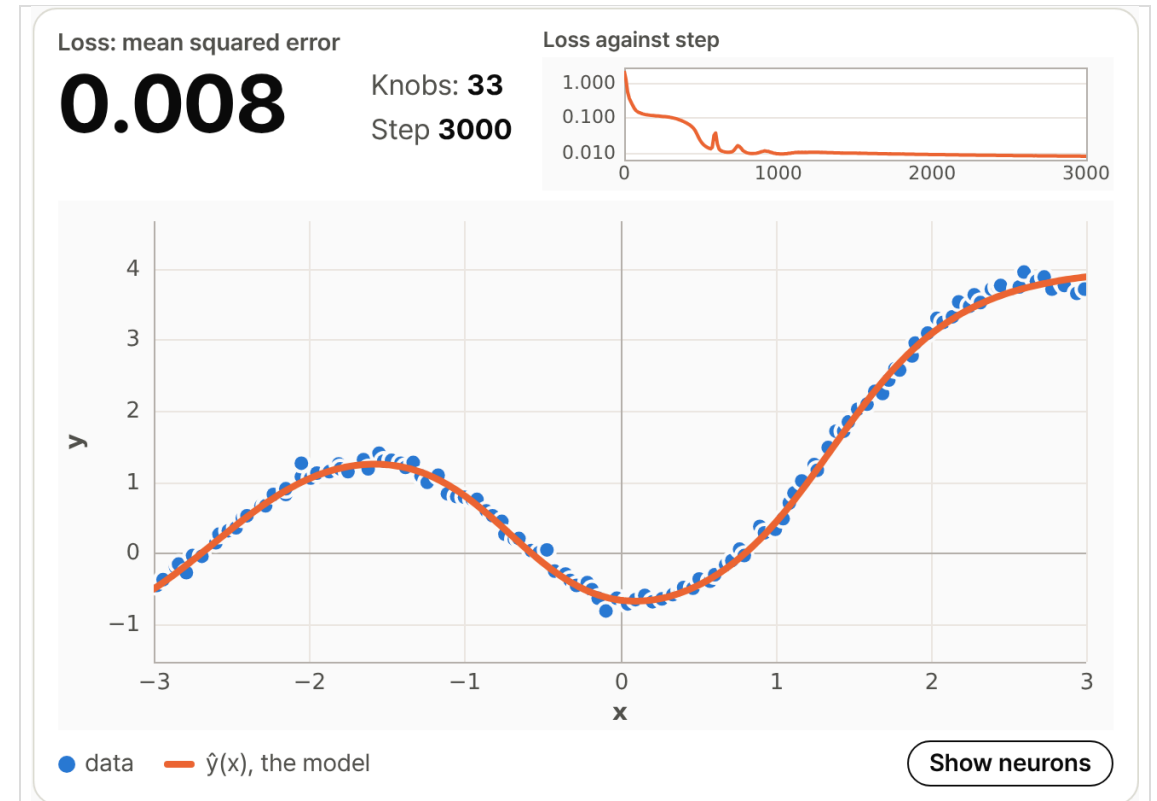
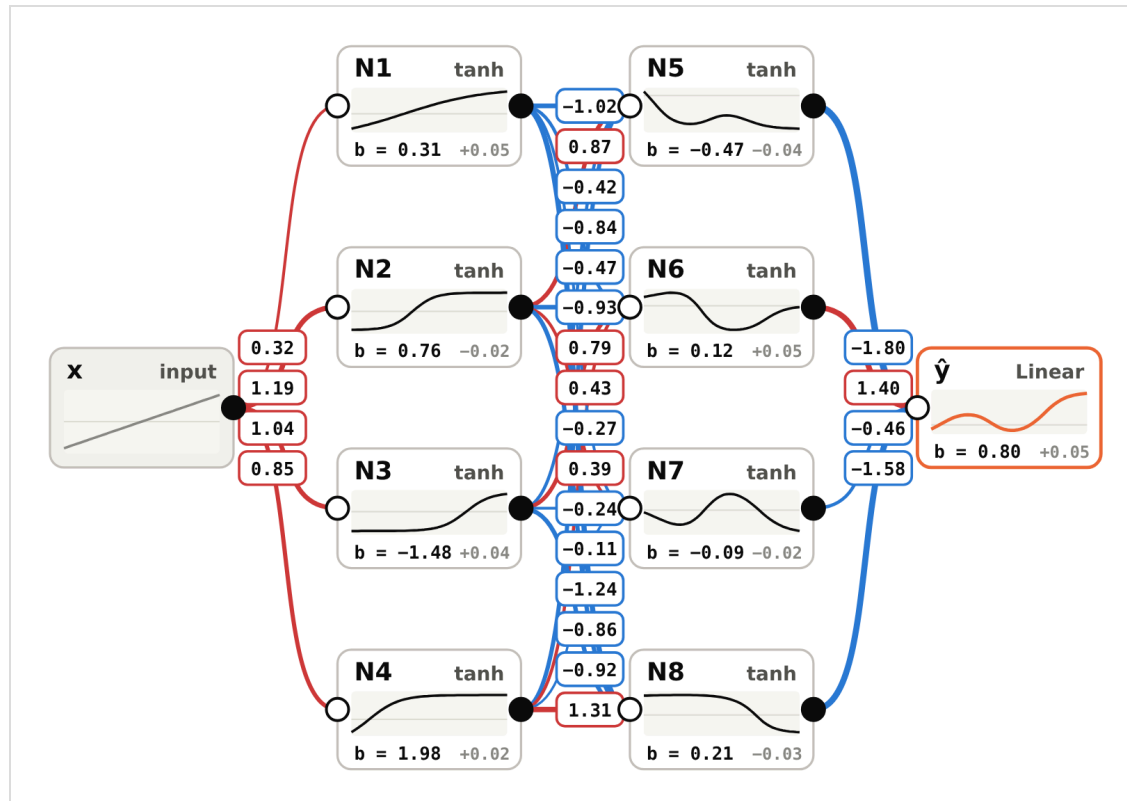
Demo, "+ Neuron", then Run



- Each ReLU neuron can add one more place where $\hat{y}$ changes its slope
- Try 1, 3, 5 and 8 neurons, and write down the loss after each Run

Two Layers

Demo, preset "2 layers": $1 \rightarrow 4 \rightarrow 4 \rightarrow 1$, tanh



- Count the knobs before you press Run: a weight on every wire, a bias on every neuron

What we have

Three Parts of Machine Learning

- A model: neurons in layers. Each neuron is a weighted sum, a bias and an activation
- A loss: one number that measures how far the predictions are from the data
- A search: gradient descent on every weight and every bias

Coming Up

- Week 4: working with LLMs, reading and verifying generated results
- Week 5: backpropagation, how every slope is measured at once

Is there a curve that enough ReLU neurons cannot fit?